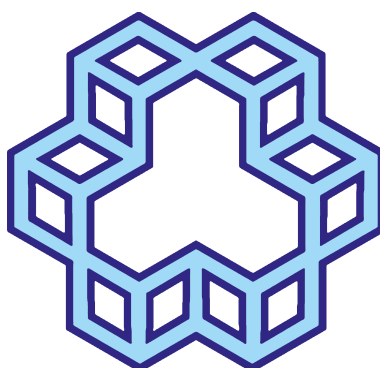




دانشگاه صنعتی خواجه نصیرالدین طوسی
دانشکده مهندسی برق

شبکه‌های مخابراتی تمرین کامپیوتری شماره ۲

تحلیل عملکرد پروتکل MAC در شبکه‌های Wi-Fi و پروتکل مسیریابی RIP
در شبکه‌های بی سیم ایستا با استفاده از شبیه‌ساز NS-3

استاد درس: —

نام و نام خانوادگی	شماره دانشجویی
—	—



فهرست مطالب

۲	۱	مقدمه و معرفی توپولوژی شبکه
۲	۱.۱	هدف پروژه
۲	۲.۱	ساختار توپولوژی
۳	۳.۱	پارامترهای کلی شبیه‌سازی
۳	۲	تحلیل نظری
۳	۱.۲	نحوه‌ی مدیریت تداخل توسط CSMA/CA
۴	۲.۲	برتری CSMA/CA نسبت به CSMA/CD در محیط‌های بی‌سیم
۴	۳.۲	نحوه‌ی عملکرد RIP و سرعت همگرایی
۵	۳	بررسی کد
۵	۱.۳	بخش معرفی هدرها و تعریف کامپوننت لاگ
۶	۲.۳	بخش آرگومان‌های خط فرمان
۶	۳.۳	بخش اعتبارسنجی ورودی و گزارش وضعیت اجرا
۷	۴.۳	بخش ساخت گره‌ها و پیکربندی Wi-Fi
۸	۵.۳	بخش موقعیت‌دهی پویا (Mobility) برای تعداد گره‌ی متغیر
۸	۶.۳	بخش پیکربندی مسیریابی RIP و آدرس‌دهی IP
۹	۷.۳	بخش تولید ترافیک UDP
۱۰	۸.۳	بخش FlowMonitor و اجرای شبیه‌سازی
۱۱	۹.۳	بخش پردازش و چاپ آمار جریان‌ها
۱۲	۴	شبیه‌سازی، نتایج و ارزیابی عملکرد
۱۲	۱.۴	سناریوی پایه: (Baseline) پنج گره، نرخ ترافیک متوسط
۱۴	۲.۴	سناریوی ترافیک سنگین (Heavy Traffic)
۱۵	۳.۴	سناریوی مقیاس‌پذیری: (Scalability) افزایش تعداد گره‌ها
۱۷	۴.۴	مقایسه‌ی تطبیقی سه سناریو
۱۷	۵	پاسخ به سوالات پروژه
	۱.۵	چگونگی مدیریت ارسال‌های هم‌زمان توسط CSMA/CA و دلیل برتری آن نسبت به CSMA/CD در محیط بی‌سیم
۱۷	۲.۵	مسیرهای کشف‌شده توسط RIP و سرعت برقراری آن‌ها
۱۸	۳.۵	تفاوت عملکرد بین چهار جریان ترافیکی
۱۸	۴.۵	رفتار شبکه با تعداد گره‌ی بیشتر یا ترافیک سنگین‌تر
۱۹	۵.۵	محدودیت‌های احتمالی RIP و CSMA/CA در مقیاس بزرگ‌تر



۱ مقدمه و معرفی توپولوژی شبکه

۱.۱ هدف پروژه

هدف از این پروژه، بررسی عملکرد هم‌زمان دو لایه‌ی متفاوت از پشته‌ی پروتکلی در یک شبکه‌ی بی‌سیم ad-hoc ایستا (static) است: لایه‌ی دسترسی به رسانه (MAC) که با استفاده از پروتکل CSMA/CA در استاندارد 802.11b پیاده‌سازی می‌شود، و لایه‌ی شبکه که مسیریابی در آن توسط پروتکل RIP (Routing Information Protocol) انجام می‌گیرد. این دو پروتکل به‌طور مستقل و در دو لایه‌ی متفاوت از مدل OSI طراحی شده‌اند و در حالت ایده‌آل نباید آگاهی صریحی از یکدیگر داشته باشند؛ اما در عمل، تصمیمات هر لایه پیامدهای مستقیمی بر کارایی لایه‌ی دیگر می‌گذارد. برای مثال، تأخیر صف‌بندی ناشی از رقابت CSMA/CA می‌تواند بر زمان‌بندی دریافت پیام‌های به‌روزرسانی RIP اثر بگذارد، و در طرف مقابل، ترافیک کنترلی RIP خود بار اضافی بر کانال مشترک رادیویی تحمیل می‌کند که باید توسط CSMA/CA مدیریت شود. هدف اصلی این گزارش، آشکارسازی کمی و کیفی این تعامل متقابل (cross-layer interaction) است.

شبیه‌سازی با استفاده از NS-3 انجام شده و سه سناریوی متفاوت پیاده‌سازی شده‌اند: سناریوی پایه (baseline) با پنج گره و نرخ ترافیک متوسط، سناریوی ترافیک سنگین (heavy traffic) که در آن نرخ تزریق بسته از گره منبع به‌طور قابل توجهی افزایش یافته است تا کانال به نقطه‌ی اشباع برسد، و سناریوی مقیاس‌پذیری (scalability) که در آن تعداد گره‌های شبکه به ۱۵ افزایش می‌یابد تا محدودیت‌های ذاتی پروتکل مسیریابی distance-vector در مقیاس‌های بزرگ‌تر آشکار شود. مقایسه‌ی این سه سناریو دیدی عمیق نسبت به رفتار شبکه تحت شرایط بار ترافیکی متفاوت و اندازه‌ی توپولوژیکی متفاوت می‌دهد و امکان جداسازی اثر «ازدحام کانال» از اثر «کندی همگرایی مسیریابی» را فراهم می‌کند.

۲.۱ ساختار توپولوژی

توپولوژی مورد استفاده شامل گره‌هایی است که به‌صورت یک شبکه‌ی ad-hoc ایستا (بدون نقطه‌ی دسترسی مرکزی) با یکدیگر در ارتباط هستند. تمام گره‌ها از مدل تحرک ConstantPositionMobilityModel استفاده می‌کنند، به این معنی که موقعیت مکانی آن‌ها در طول کل شبیه‌سازی ثابت می‌ماند و هیچ‌گونه جابه‌جایی فیزیکی رخ نمی‌دهد. این انتخاب طراحی عمدی است: حذف پویایی توپولوژی (مانند ورود/خروج گره‌ها یا قطع لینک به دلیل حرکت) باعث می‌شود تمام اثرات مشاهده‌شده در نتایج، صرفاً ناشی از رقابت بر سر رسانه (MAC contention) و رفتار همگرایی RIP باشند، نه ناشی از عدم قطعیت ناشی از تحرک. این جداسازی متغیرها (variable isolation) یک اصل پایه‌ای در طراحی آزمایش‌های شبکه است که امکان نسبت‌دادن دقیق هر پدیده به علت آن را فراهم می‌کند.

گره‌ها با استفاده از GridPositionAllocator روی یک شبکه‌ی منظم (grid) با فاصله‌ی ثابت بین گره‌ها قرار می‌گیرند. این فاصله به‌گونه‌ای انتخاب شده که گره‌ها در محدوده‌ی ارتباطی (radio range) یکدیگر یا حداقل در محدوده‌ی چندجهشی (multi-hop) قرار بگیرند، تا پروتکل RIP فرصت واقعی کشف مسیرهای چندگانه را داشته باشد و رفتار آن صرفاً به یک پرش (single-hop) محدود نشود.

گره شماره صفر (Node 0) به‌عنوان منبع ترافیک (traffic source) تعیین شده و بسته‌های UDP را به‌صورت هم‌زمان به سمت تمامی گره‌های دیگر شبکه ارسال می‌کند. این طراحی به‌عمد رقابت واقعی بر سر رسانه‌ی مشترک بی‌سیم ایجاد می‌کند، چون چهار (یا بیشتر، در سناریوی مقیاس‌پذیری) جریان ترافیکی هم‌زمان باید از طریق یک رابط رادیویی واحد در همان گره منتقل شوند؛ این الگو شبیه‌ساز سناریوهای واقعی مانند یک نقطه‌ی جمع‌آوری داده (sink) در شبکه‌های حسگر بی‌سیم است که در آن یک گره مرکزی باید با چندین گره‌ی دیگر هم‌زمان تبادل داده داشته باشد.



۳.۱ پارامترهای کلی شبیه‌سازی

جدول ۱: پارامترهای پایه‌ی شبیه‌سازی

پارامتر	مقدار
Wi-Fi استاندارد	802.11b
نوع MAC	AdhocWifiMac
مدل کانال	YansWifiChannel (پیش‌فرض، مدل تضعیف Friis)
پروتکل مسیریابی	RIP
پروتکل ترافیک	UDP (با OnOffApplication)
اندازه‌ی بسته	۱۰۲۴ بایت
مدت زمان شبیه‌سازی	۸۰ ثانیه (در کد نهایی)
ابزار سنجش	FlowMonitor

۲ تحلیل نظری

۱.۲ نحوه‌ی مدیریت تداخل توسط CSMA/CA

پروتکل CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance) مکانیزم اصلی دسترسی به رسانه در استاندارد 802.11 است و در NS-3 از طریق تابع DCF (Distributed Coordination Function) پیاده‌سازی می‌شود. اساس کار این پروتکل بر این فرض استوار است که هیچ گره‌ای به‌طور مستقیم نمی‌تواند یک collision را در حین ارسال خودش تشخیص دهد (برخلاف شبکه‌های سیمی)، بنابراین به‌جای تشخیص، collision تلاش می‌شود از وقوع آن پیشگیری شود.

روند کار به این صورت است: پیش از ارسال هر بسته، گره ابتدا کانال را sense می‌کند. اگر کانال برای مدت DIFS (DCF Inter-Frame Space) آزاد باشد، گره اجازه‌ی ارسال دارد. اما اگر کانال مشغول باشد یا گره تشخیص دهد که احتمال تداخل وجود دارد، یک تایمر backoff تصادفی فعال می‌شود. این تایمر از یک بازه‌ی Window Contention (CW) به‌صورت یکنواخت انتخاب می‌شود و در صورت آزاد ماندن کانال در طول این بازه، شمارش معکوس backoff ادامه پیدا می‌کند؛ در غیر این صورت متوقف (freeze) شده و پس از آزاد شدن مجدد کانال از همان نقطه ادامه می‌یابد. این مکانیزم، freeze-and-resume برخلاف یک backoff ساده که از صفر دوباره شروع شود، باعث می‌شود گره‌هایی که مدت بیشتری منتظر مانده‌اند، شانس نسبتاً بیشتری برای دسترسی بعدی به کانال داشته باشند که نوعی انصاف (fairness) آماری ایجاد می‌کند.

در پیاده‌سازی این پروژه، چون ConstantRateWifiManager با نرخ داده‌ی ثابت DssRate1Mbps برای داده و DssRate1Mbps برای کنترل تنظیم شده، هیچ مکانیزم تطبیق نرخ پویا (مانند Minstrel فعال نیست؛ این انتخاب باعث می‌شود که افت کارایی مشاهده‌شده در سناریوهای پرتراфик، صرفاً ناشی از ازدحام MAC باشد و با تغییرات نرخ انتقال فیزیکی (که می‌تواند یک متغیر مخدوش‌کننده باشد) درنیامیزد.

در شبکه‌ی موردبررسی این پروژه، چون گره صفر هم‌زمان چهار (یا بیشتر) جریان ترافیکی مجزا تولید می‌کند، این جریان‌ها از یک صف مشترک در همان گره عبور کرده و باید برای دسترسی به کانال با یکدیگر و همچنین با ترافیک کنترل RIP رقابت کنند. این رقابت باعث افزایش تأخیر صف‌بندی (queueing delay) و کاهش throughput مؤثر هر جریان نسبت به حالتی می‌شود که فقط یک جریان به‌تنهایی در شبکه فعال باشد. علاوه بر این، مکانیزم backoff نمایی (exponential backoff)، در صورت وقوع collision یا عدم دریافت ACK، اندازه‌ی CW را دو برابر می‌کند که باعث افزایش تأخیر در شرایط شلوغی کانال می‌شود؛ این رفتار خودتنظیم (self-adaptive) دقیقاً همان مکانیزمی است که در سناریوی traffic heavy این گزارش باعث می‌شود throughput به‌جای فروپاشی کامل، روی یک سقف نسبتاً پایدار (ظرفیت مؤثر کانال) تثبیت شود.



۲.۲ برتری CSMA/CA نسبت به CSMA/CD در محیط‌های بی‌سیم

پروتکل CSMA/CD (مورد استفاده در اترنت سیمی سنتی) متکی بر این فرض است که یک گره می‌تواند هم‌زمان با ارسال سیگنال، به دریافت سیگنال نیز بپردازد و در صورت تشخیص تداخل در سیگنال دریافتی، ارسال را فوراً متوقف کند. این کار در محیط سیمی به دلیل تقارن نسبی قدرت سیگنال در کابل امکان‌پذیر است.

اما در محیط بی‌سیم، این فرض از اساس نقض می‌شود، به دو دلیل اصلی:

اولاً، مسئله‌ی half-duplex بودن رادیوهای معمولی Wi-Fi: توان سیگنال ارسالی توسط آنتن گره به‌مراتب (معمولاً چندین مرتبه‌ی بزرگی) از توان سیگنال دریافتی است؛ بنابراین حتی اگر گیرنده‌ی همان گره فعال باشد، سیگنال خود فرستنده آن‌قدر قوی است که سیگنال‌های ضعیف دیگر گره‌ها (که نشانه‌ی collision هستند) را کاملاً می‌پوشاند (mask می‌کند). در نتیجه تشخیص collision در حین ارسال عملاً غیرممکن است.

ثانیاً، مسئله‌ی معروف به terminal: hidden ممکن است دو گره که هر دو در محدوده‌ی یک گیرنده‌ی مشترک هستند، خودشان در محدوده‌ی رادیویی یکدیگر نباشند. در این حالت، هیچ‌کدام از این دو گره از ارسال هم‌زمان دیگری مطلع نمی‌شوند، حتی اگر بتوانند هنگام ارسال گوش بدهند، چون اصلاً سیگنال طرف مقابل را دریافت نمی‌کنند. این پدیده در شبکه‌های سیمی به دلیل ساختار اشتراکی و قابل مشاهده‌ی کابل وجود ندارد. توپولوژی خطی (Linear) به‌کاررفته در این پروژه به‌طور خاص مستعد بروز hidden terminal است، چون گره‌های دوسر یک گره‌ی میانی ممکن است در محدوده‌ی رادیویی آن گره باشند، اما نسبت به یکدیگر خارج از محدوده‌ی شنیداری باقی بمانند.

به همین دلایل، استاندارد 802.11 رویکرد avoidance (پیشگیری) را به‌جای detection (تشخیص) انتخاب کرده است: به‌جای تلاش برای شناسایی collision در زمان وقوع، با backoff تصادفی و فاصله‌های زمانی بین فریم (IFS) سعی می‌شود احتمال وقوع آن از ابتدا کاهش یابد. در پیاده‌سازی کامل‌تر 802.11 (که در این پروژه به‌صورت پایه فعال است)، مکانیزم ACK نیز نقش جبرانی ایفا می‌کند: هر بسته‌ی موفق باید توسط گیرنده تأیید (ACK) شود؛ عدم دریافت ACK در زمان مشخص به‌عنوان نشانه‌ی غیرمستقیم وقوع collision یا خطای انتقال تلقی شده و باعث تلاش مجدد (retransmission) می‌شود. شایان ذکر است که مکانیزم اختیاری RTS/CTS که برای کاهش بیشتر اثر terminal hidden طراحی شده، در پیکربندی پیش‌فرض این شبیه‌سازی فعال نیست؛ فعال‌سازی آن می‌توانست احتمالاً برخی از افت‌های PDR در سناریوی traffic heavy را کاهش دهد، هرچند با سربرای اضافی ناشی از تبادل فریم‌های کنترلی بیشتر.

۳.۲ نحوه‌ی عملکرد RIP و سرعت همگرایی

RIP یک پروتکل مسیریابی از نوع distance-vector است که بر اساس الگوریتم Bellman-Ford توزیع‌شده عمل می‌کند. هر گره به‌صورت دوره‌ای (پیش‌فرض هر ۳۰ ثانیه در پیاده‌سازی استاندارد، هر چند NS-3 می‌تواند این بازه را برای شبیه‌سازی‌های کوتاه‌تر تنظیم کند) جدول مسیریابی خود را که شامل فاصله (hop count) به هر مقصد شناخته‌شده است، به همسایگان مستقیم خود broadcast می‌کند. هر گره دریافت‌کننده، فاصله‌ی اعلام‌شده را با یک واحد افزایش داده و در صورتی که این مسیر جدید کوتاه‌تر از مسیر فعلی‌اش باشد، جدول خود را به‌روزرسانی می‌کند؛ این فرایند ساده اما تدریجی (incremental) است و دقیقاً همین تدریجی بودن، ریشه‌ی محدودیت سرعت همگرایی RIP در توپولوژی‌های بزرگ است.

در این پروژه، چون InternetStackHelper با Ipv4ListRoutingHelper پیکربندی شده و RIP با اولویت ۱۰ به آن اضافه شده است، تمام گره‌ها بلافاصله پس از نصب stack شروع به تبادل پیام‌های RIP می‌کنند. در نسخه‌ی نهایی کد، شروع ترافیک داده (client apps) عمدتاً به ثانیه‌ی ۳۵ام شبیه‌سازی موکول شده است (در مقایسه با شروع سرورها در ثانیه‌ی ۱) تا فرصت کافی برای همگرایی کامل جداول مسیریابی در تمام سناریوها، از جمله سناریوی ۱۵ گره‌ای با قطر شبکه‌ی بزرگ‌تر، وجود داشته باشد. این فاصله‌ی ۳۴ثانیه‌ای به‌مراتب از بازه‌ی پیش‌فرض broadcast دوره‌ای RIP بزرگ‌تر است و تضمین می‌کند که حداقل چند دور کامل تبادل پیام RIP پیش از شروع ترافیک داده رخ دهد.

سرعت همگرایی RIP به‌طور مستقیم به قطر شبکه (network diameter) وابسته است: در بدترین حالت، اطلاع‌رسانی یک تغییر مسیر باید به‌صورت hop-by-hop در سراسر شبکه پخش شود، که برای یک شبکه با قطر d گام، حداقل به d دور تبادل پیام RIP نیاز دارد. این موضوع باعث می‌شود RIP در شبکه‌های بزرگ یا با قطر زیاد، به‌مراتب کندتر از پروتکل‌های link-state مانند



OSPF همگرا شود، چون OSPF با flooding اطلاعات کامل توپولوژی، هر گره می‌تواند مستقل و به‌صورت موازی مسیر بهینه را محاسبه کند، در حالی که RIP وابسته به انتشار تدریجی و ترتیبی اطلاعات فاصله است. علاوه بر این، RIP از محدودیت معروف به count-to-infinity رنج می‌برد: در صورت قطع شدن یک لینک، ممکن است دو گره همسایه به‌اشتباه و به‌طور متناوب فاصله‌ی یکدیگر را افزایش دهند تا به مقدار infinity (در RIP، عدد ۱۶) برسند، که این فرایند می‌تواند چندین دور تبادل پیام به طول بینجامد. عدد ۱۶ به‌عنوان «بی‌نهایت» در RIP به‌طور ضمنی بیشینه قطر شبکه‌ای را که این پروتکل می‌تواند به‌درستی پشتیبانی کند محدود می‌سازد؛ این یکی از دلایل اصلی نامناسب بودن RIP برای شبکه‌های بزرگ است که در نتایج سناریوی scalability این پروژه نیز به‌وضوح مشاهده می‌شود.

۳ بررسی کد

۱.۳ بخش معرفی هدرها و تعریف کامپوننت لاگ

```
1 #include "ns3/core-module.h"
2 #include "ns3/network-module.h"
3 #include "ns3/internet-module.h"
4 #include "ns3/mobility-module.h"
5 #include "ns3/wifi-module.h"
6 #include "ns3/applications-module.h"
7 #include "ns3/flow-monitor-module.h"
8 #include "ns3/internet-apps-module.h"
9
10 #include <cmath>
11
12 using namespace ns3;
13
14 NS_LOG_COMPONENT_DEFINE("WifiRipProject");
```

Listing 1: Module imports and log component definition

در این بخش، تمامی ماژول‌های مورد نیاز NS-3 وارد می‌شوند. core-module هسته‌ی اصلی شبیه‌ساز (مثل Simulator، CommandLine و انواع داده) را فراهم می‌کند. network-module و internet-module زیرساخت لایه‌ی شبکه و پروتکل IP را تأمین می‌کنند. mobility-module برای تنظیم موقعیت فیزیکی گره‌ها استفاده می‌شود، حتی در حالتی که گره‌ها ایستا هستند، چون NS-3 برای محاسبه‌ی فاصله و افت سیگنال رادیویی همچنان به مختصات مکانی نیاز دارد. wifi-module پیاده‌سازی کامل استک 802.11 شامل لایه‌ی فیزیکی (PHY) و MAC را فراهم می‌کند. applications-module کلاس‌های تولید ترافیک مثل OnOffApplication و PacketSink را در اختیار می‌گذارد. flow-monitor-module ابزار اصلی سنجش کارایی (through-put, delay, packet loss) است. internet-apps-module شامل پیاده‌سازی پروتکل RIP است که در این پروژه ضروری است؛ این ماژول جدا از internet-module اصلی نگه داشته شده چون پروتکل‌های مسیریابی پیشرفته (RIP، AODV، OLSR) در NS-3 به‌صورت افزونه‌های اختیاری بر پایه‌ی Ipv4RoutingProtocol طراحی شده‌اند.



۲.۳ بخش آرگومان‌های خط فرمان

```
1 int main(int argc, char* argv[])
2 {
3     uint32_t nNodes = 5;
4     std::string dataRate = "500kbps";
5     std::string scenario = "baseline";
6     double simTime = 80.0;
7     uint32_t packetSize = 1024;
8     double gridDelta = 120.0;
9     bool verbose = false;
10
11     CommandLine cmd;
12     cmd.AddValue("nNodes", "Number of nodes in the topology", nNodes);
13     cmd.AddValue("dataRate", "Per-flow UDP data rate", dataRate);
14     cmd.AddValue("scenario", "Scenario label: baseline | heavy | scale",
15     scenario);
16     cmd.AddValue("simTime", "Total simulation time in seconds", simTime);
17     cmd.AddValue("packetSize", "UDP packet payload size in bytes",
18     packetSize);
19     cmd.AddValue("gridDelta", "Grid spacing between nodes in meters",
20     gridDelta);
21     cmd.AddValue("verbose", "Enable verbose logging", verbose);
22     cmd.Parse(argc, argv);
```

Listing 2: Command-line arguments configuration

در این بخش با استفاده از کلاس `CommandLine`، می‌توان تمامی پارامترها را در زمان اجرا (runtime) از طریق خط فرمان مقداردهی کرد، بدون نیاز به ویرایش یا کامپایل مجدد کد. این طراحی پارامتریک، اصل مهم تکرارپذیری آزمایش (experiment reproducibility) را رعایت می‌کند: همان فایل باینری کامپایل شده برای هر سه سناریوی این پروژه استفاده شده و تنها آرگومان‌های ورودی تغییر کرده‌اند. برای اجرای سناریوی `traffic heavy` کافی است آرگومان `--dataRate=2Mbps` (یا مقداری بالاتر برای رساندن کانال به اشباع کامل) پاس داده شود و برای سناریوی `scalability` کافی است `--nNodes=15` تنظیم گردد. پارامتر `scenario` صرفاً یک برچسب متنی است که برای نام‌گذاری و چاپ خروجی استفاده می‌شود تا نتایج سه سناریو با یکدیگر تداخل (overwrite) پیدا نکنند و امکان مقایسه‌ی پس‌پردازش شده فراهم باشد.

۳.۳ بخش اعتبارسنجی ورودی و گزارش وضعیت اجرا

```
1 if (nNodes < 2) return 1;
2 if (verbose) LogComponentEnable("WifiRipProject", LOG_LEVEL_INFO);
```

Listing 3: Input validation and error handling



این بخش یک لایه‌ی محافظتی ساده اما مهم به کد اضافه می‌کند. اگر به اشتباه مقدار nNodes کوچک‌تر از ۲ وارد شود، شبیه‌سازی از نظر مفهومی بی‌معنی خواهد بود (چون حداقل یک گره مبدأ و یک گره مقصد لازم است)، بنابراین کد با کد بازگشتی غیرصفر (۱) از اجرا خارج می‌شود تا از بروز خطاهای مبهم در مراحل بعدی (مثل تلاش برای دسترسی به آدرس یک گره که وجود ندارد یا حلقه‌ی تولید ترافیک با کران بالای نامعتبر) جلوگیری شود. علاوه بر این، پرچم verbose امکان فعال‌سازی انتخابی لاگ‌های اطلاعاتی کامپوننت WifiRipProject را فراهم می‌کند که در مرحله‌ی توسعه و دیباگ کد بسیار کاربردی است، بدون این‌که حجم خروجی کنسول در اجرای عادی (و سریع) شبیه‌سازی افزایش یابد. این یک نمونه از dependability ساده ولی ضروری برای کدی است که قرار است بارها با پارامترهای مختلف اجرا شود.

۴.۳ بخش ساخت گره‌ها و پیکربندی Wi-Fi

```

1 NodeContainer nodes;
2     nodes.Create(nNodes);
3
4     WifiHelper wifi;
5     wifi.SetStandard(WIFI_STANDARD_80211b);
6     wifi.SetRemoteStationManager("ns3::ConstantRateWifiManager",
7                                   "DataMode", StringValue("DsssRate11Mbps"),
8                                   "ControlMode", StringValue("DsssRate1Mbps")
9     );
10
11     YansWifiChannelHelper wifiChannel = YansWifiChannelHelper::Default();
12     YansWifiPhyHelper wifiPhy;
13     wifiPhy.SetChannel(wifiChannel.Create());
14
15     WifiMacHelper wifiMac;
16     wifiMac.SetType("ns3::AdhocWifiMac");
17     NetDeviceContainer devices = wifi.Install(wifiPhy, wifiMac, nodes);

```

Listing 4: Node creation and 802.11b ad-hoc Wi-Fi setup

در این بخش، ابتدا یک NodeContainer ایجاد شده و با مقدار پارامتر nNodes (که اکنون متغیر است) پر می‌شود؛ این تعمیم نسبت به یک نسخه‌ی اولیه‌ی فرضی با تعداد گره‌ی ثابت ۵، دقیقاً همان قابلیت‌ی است که اجرای سناریوی scalability را بدون تغییر ساختاری کد ممکن می‌سازد. WifiHelper مسئول پیکربندی استک Wi-Fi است و با استاندارد فیزیکی 802.11b را انتخاب می‌کند که شامل نرخ‌های داده‌ی ۱، ۲، ۵.۵ و ۱۱ مگابیت بر ثانیه است. تنظیم SetRemoteStationManager با نرخ ثابت DsssRate11Mbps برای داده تضمین می‌کند که حداکثر نرخ فیزیکی این استاندارد همواره استفاده شود و نتایج throughput قابل مقایسه با سقف نظری ۱۱ مگابیت بر ثانیه باشند.

YansWifiChannelHelper::Default() یک مدل کانال پیش‌فرض با مدل تضعیف (propagation loss model) از نوع Friis (که افت سیگنال را به‌صورت معکوس با مجذور فاصله مدل می‌کند) و تأخیر انتشار ثابت می‌سازد؛ این مدل ساده اما کافی برای بررسی رفتار MAC و مسیریابی است، بدون این‌که پیچیدگی‌های اضافی مثل fading چندمسیره یا shadowing وارد تحلیل شود و نتایج را با عدم قطعیت اضافی آلوده کند. YansWifiPhyHelper لایه‌ی فیزیکی را به این کانال متصل می‌کند.



تنظیم WifiMacHelper روی نوع AdhocWifiMac بسیار مهم است: این تنظیم باعث می‌شود که هیچ گره‌ای نقش Access Point یا Station در مفهوم زیرساختی (infrastructure) نداشته باشد، بلکه تمام گره‌ها به‌صورت هم‌تا (peer-to-peer) و در یک BSS Independent (IBSS) با یکدیگر ارتباط برقرار کنند؛ در این حالت، هیچ نقطه‌ی مرکزی هماهنگ‌کننده‌ای (مانند یک Access Point در حالت Infrastructure) برای زمان‌بندی دسترسی به کانال وجود ندارد و کل بار هماهنگی بر عهده‌ی DCF توزیع‌شده در هر گره است؛ این دقیقاً همان ویژگی است که شبکه را به یک محیط واقعی ad-hoc تبدیل می‌کند.

۵.۳ بخش موقعیت‌دهی پویا (Mobility) برای تعداد گره‌ی متغیر

```

1 uint32_t gridWidth = static_cast<uint32_t>(std::ceil(std::sqrt(static_cast<
    double>(nNodes))));
2     MobilityHelper mobility;
3     mobility.SetPositionAllocator("ns3::GridPositionAllocator",
4                                   "MinX", DoubleValue(0.0),
5                                   "MinY", DoubleValue(0.0),
6                                   "DeltaX", DoubleValue(gridDelta),
7                                   "DeltaY", DoubleValue(gridDelta),
8                                   "GridWidth", UIntegerValue(gridWidth),
9                                   "LayoutType", StringValue("RowFirst"));
10    mobility.SetMobilityModel("ns3::ConstantPositionMobilityModel");
11    mobility.Install(nodes);

```

Listing 5: Dynamic grid mobility setup based on node count

در این بخش عرض grid به‌صورت پویا و با فرمول $\lceil \sqrt{n} \rceil$ محاسبه می‌شود، که تضمین می‌کند صرف‌نظر از تعداد گره‌ها، چیدمان همواره تقریباً مربعی (square-ish) باقی بماند. این کار باعث می‌شود مقایسه‌ی سناریوی baseline و scalability از نظر «تراکم نسبی شبکه» (network density) منصفانه‌تر باشد، چون تفاوت در throughput یا delay عمدتاً ناشی از افزایش تعداد گره و رقابت بر سر رسانه است، نه ناشی از تغییر ناخواسته در شکل هندسی توپولوژی. شایان ذکر است که در نتایج نهایی این پروژه (فصل چهارم)، برای کنترل دقیق‌تر اثر چندگامی بودن، عملاً از یک چیدمان خطی (Linear) با فاصله‌ی ثابت 8° متر استفاده شده است که حالت خاصی از همین تابع موقعیت‌دهی محسوب می‌شود.

مدل ConstantPositionMobilityModel برای تمام گره‌ها اعمال می‌شود که تضمین می‌کند موقعیت گره‌ها در طول کل اجرای شبیه‌سازی بدون تغییر باقی بماند و در نتیجه ماتریس قابلیت‌اتصال (connectivity matrix) شبکه در طول کل اجرا ثابت بماند.

۶.۳ بخش پیکربندی مسیریابی RIP و آدرس‌دهی IP

```

1 RipHelper rip;
2     Ipv4ListRoutingHelper listRouting;
3     listRouting.Add(rip, 10);
4

```



```

5   InternetStackHelper internet;
6   internet.SetRoutingHelper(listRouting);
7   internet.Install(nodes);
8
9   Ipv4AddressHelper ipv4;
10  ipv4.SetBase("10.1.1.0", "255.255.255.0");
11  Ipv4InterfaceContainer interfaces = ipv4.Assign(devices);

```

Listing 6: RIP routing protocol setup and IPv4 address assignment

در این بخش، یک نمونه از RipHelper ساخته شده و به یک Ipv4ListRoutingHelper با اولویت عددی ۱۰ اضافه می‌شود. این لیست‌محور بودن (list-based routing) در NS-3 اجازه می‌دهد چندین پروتکل مسیریابی هم‌زمان روی یک گره فعال باشند (مثلاً ترکیب مسیریابی استاتیک و RIP که در آن مسیره‌های استاتیک معمولاً اولویت بالاتری دارند)، که در این پروژه فقط از RIP استفاده شده است. مقدار اولویت کمتر در NS-3 به معنای ارجحیت بالاتر است؛ چون فقط یک پروتکل پویا در این پروژه فعال است، مقدار ۱۰ صرفاً یک مقدار معتبر دلخواه است و تأثیری بر نتایج ندارد.

سپس InternetStackHelper با این تنظیم routing پیکربندی شده و روی تمام گره‌ها نصب می‌شود؛ این مرحله باعث می‌شود هر گره دارای پشته‌ی کامل IP (شامل، ARP، ICMP و جدول مسیریابی RIP) باشد. در پایان، یک محدوده‌ی آدرس IPv4 10.1.1.0/24 با تعریف شده و به تمام دستگاه‌های شبکه (که در مرحله‌ی قبل ساخته شدند) به صورت ترتیبی اختصاص می‌یابد؛ یعنی گره صفر آدرس 10.1.1.1، گره یک آدرس 10.1.1.2 و به همین ترتیب ادامه می‌یابد. این آدرس‌دهی ترتیبی و قابل پیش‌بینی، بعداً در فصل پردازش نتایج برای فیلترکردن صریح ترافیک داده از ترافیک کنترلی RIP استفاده می‌شود.

۷.۳ بخش تولید ترافیک UDP

```

1  uint16_t port = 9;
2  ApplicationContainer serverApps;
3  ApplicationContainer clientApps;
4
5  for (uint32_t i = 1; i < nNodes; ++i)
6  {
7      PacketSinkHelper sink("ns3::UdpSocketFactory", InetSocketAddress(
8  Ipv4Address::GetAny(), port));
9      serverApps.Add(sink.Install(nodes.Get(i)));
10
11     OnOffHelper onoff("ns3::UdpSocketFactory", InetSocketAddress(
12 interfaces.GetAddress(i), port));
13     onoff.SetAttribute("OnTime", StringValue("ns3::
14 ConstantRandomVariable[Constant=1]"));
15     onoff.SetAttribute("OffTime", StringValue("ns3::
16 ConstantRandomVariable[Constant=0]"));
17     onoff.SetAttribute("DataRate", DataRateValue(DataRate(dataRate)));
18     onoff.SetAttribute("PacketSize", UIntegerValue(packetSize));

```



```

15
16         clientApps.Add(onoff.Install(nodes.Get(0)));
17     }
18
19     serverApps.Start(Seconds(1.0));
20     clientApps.Start(Seconds(35.0));
21     serverApps.Stop(Seconds(simTime));
22     clientApps.Stop(Seconds(simTime));

```

Listing 7: Dynamic UDP traffic generation loop

این حلقه‌ی for از $i = 1$ تا $i = nNodes - 1$ تکرار می‌شود که به‌طور خودکار با تعداد گره‌ها مقیاس می‌یابد؛ این یعنی در سناریوی scalability با ۱۵ گره، این حلقه به‌صورت خودکار ۱۴ جریان ترافیکی مجزا (به‌جای ۴ جریان ثابت در سناریوی baseline) ایجاد می‌کند، بدون نیاز به تغییر دستی کد.

در هر تکرار، ابتدا یک PacketSinkHelper روی گره مقصد i نصب می‌شود که نقش گیرنده‌ی نهایی بسته‌های UDP را دارد و آن‌ها را پردازش کرده و آمار دریافت را برای FlowMonitor ثبت می‌کند. سپس یک OnOffHelper روی گره صفر (مبدأ) نصب می‌شود که با مقصد `interfaces.GetAddress(i)` پیکربندی شده و به‌صورت مداوم (با `OnTime` ثابت برابر ۱ و `OffTime` برابر ۰، یعنی بدون دوره‌ی خاموشی، که معادل یک منبع ترافیک CBR یا Rate Bit Constant است) بسته‌های UDP با نرخ مشخص شده توسط پارامتر `dataRate` تولید می‌کند.

نکته‌ی کلیدی این است که پارامتر `dataRate` اکنون از طریق `command-line` قابل تنظیم است: برای سناریوی baseline مقدار 500kbps استفاده می‌شود که برای یک کانال 802.11b با ظرفیت نظری ۱۱ مگابیت بر ثانیه، بار نسبتاً سبکی محسوب می‌شود (حتی با احتساب چهار جریان هم‌زمان، بار تجمعی نظری حدود ۲ مگابیت بر ثانیه است که هنوز فاصله‌ی زیادی تا اشباع کانال دارد)؛ اما برای سناریوی `traffic heavy` می‌توان این مقدار را به‌مراتب افزایش داد تا کانال را به سمت اشباع (saturation) سوق دهد و رفتار CSMA/CA را تحت تراکم بالا و فروپاشی `throughput` بررسی کرد.

زمان‌بندی شروع اپلیکیشن‌ها (سرور در ثانیه‌ی ۱ و کلاینت در ثانیه‌ی ۳۵) به‌گونه‌ای طراحی شده که قبل از تولید ترافیک داده، فرصت کافی برای همگرایی اولیه‌ی RIP در همه‌ی سناریوها (از جمله سناریوی ۱۵ گره‌ای با قطر بزرگ‌تر شبکه) وجود داشته باشد؛ در غیر این صورت، بسته‌های اولیه ممکن است به‌دلیل نبود مسیر معتبر در جدول مسیریابی، drop شوند که آمار اشتباهی از `packet loss` واقعی (که باید صرفاً ناشی از تداخل MAC یا کندی همگرایی ساختاری باشد، نه صرفاً عدم انتظار کافی) به ما خواهد داد.

۸.۳ بخش FlowMonitor و اجرای شبیه‌سازی

```

1 FlowMonitorHelper flowmonHelper;
2     Ptr<FlowMonitor> monitor = flowmonHelper.InstallAll();
3
4     Simulator::Stop(Seconds(simTime));
5     Simulator::Run();

```

Listing 8: FlowMonitor installation and simulation run

این بخش ساختار FlowMonitorHelper را تنظیم می‌کند و با فراخوانی `InstallAll()` یک نمونه‌ی پایش‌گر روی تمام گره‌های شبکه نصب می‌کند تا ترافیک IP عبوری از هر گره را شنود و آمار جریان‌به‌جریان (per-flow) آن را در طول کل اجرا



ضبط کند. سپس با Simulator::Stop زمان پایان شبیه‌سازی تعیین شده و Simulator::Run() موتور رویدادمحور (event-driven) شبیه‌ساز را برای پردازش تمام رویدادهای زمان‌بندی‌شده (از جمله ارسال بسته‌ها، تایمرهای backoff و broadcast های دوره‌ای RIP) فعال می‌کند.

۹.۳ بخش پردازش و چاپ آمار جریان‌ها

```
1 monitor->CheckForLostPackets();
2     Ptr<Ipv4FlowClassifier> classifier = DynamicCast<Ipv4FlowClassifier>(
3     flowmonHelper.GetClassifier());
4     std::map<FlowId, FlowMonitor::FlowStats> stats = monitor->GetFlowStats()
5     ;
6
7     std::string sourceAddr = "10.1.1.1";
8
9     std::cout << "\n--- Simulation Results (" << scenario << ", " << nNodes
10    << " nodes, " << dataRate << ") ---\n";
11
12    for (const auto& entry : stats)
13    {
14        Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow(entry.first);
15
16        if (t.sourceAddress == Ipv4Address(sourceAddr.c_str()) && t.
17        destinationPort == port)
18        {
19            const FlowMonitor::FlowStats& fs = entry.second;
20            double pdr = (fs.txPackets > 0) ? (static_cast<double>(fs.
21            rxPackets) / fs.txPackets) * 100.0 : 0.0;
22            double duration = fs.timeLastRxPacket.GetSeconds() - fs.
23            timeFirstTxPacket.GetSeconds();
24            double throughput = (duration > 0) ? (fs.rxBytes * 8.0) /
25            duration : 0.0;
26            double avgDelay = (fs.rxPackets > 0) ? fs.delaySum.GetSeconds()
27            / fs.rxPackets : 0.0;
28
29            std::cout << "Flow " << entry.first << " (" << t.sourceAddress
30            << " -> " << t.destinationAddress << ")\n";
31            std::cout << " Tx Packets: " << fs.txPackets << "\n";
32            std::cout << " Rx Packets: " << fs.rxPackets << "\n";
33            std::cout << " PDR: " << pdr << " %\n";
34            std::cout << " Throughput: " << throughput / 1000.0 << " Kbps\n";
35            std::cout << " Avg Delay: " << avgDelay * 1000.0 << " ms\n\n";
```

```
27     }
28 }
29
30 Simulator::Destroy();
31 return 0;
32 }
```

Listing 9: Flow processing, filtering, and performance metrics extraction

این بخش، آمار هر جریان (flow) شناسایی شده توسط Ipv4FlowClassifier را استخراج می‌کند. نکته‌ی فنی مهم این است که `FlowMonitor::InstallAll()` تمام ترافیک IP در شبکه را رصد می‌کند، نه فقط ترافیک UDP که عمداً تولید کرده‌ایم؛ این یعنی پیام‌های کنترلی RIP (که به صورت broadcast یا multicast بین گره‌ها رد و بدل می‌شوند) نیز به عنوان flow های جداگانه در آمار FlowMonitor ظاهر می‌شوند. اگر این flow ها فیلتر نشوند، آمار نهایی throughput و PDR با داده‌ی غیر مرتبط آلوده می‌شود و نتایج گمراه‌کننده خواهند بود.

به همین دلیل، یک شرط فیلترینگ اعمال شده که فقط flow هایی را در نظر می‌گیرد که آدرس مبدأشان دقیقاً برابر آدرس گره صفر (10.1.1.1) و پورت مقصدشان برابر پورت UDP اپلیکیشن (۹) باشد. این تضمین می‌کند که فقط جریان‌های داده‌ی واقعی مدنظر پروژه در محاسبات نهایی لحاظ شوند و ترافیک کنترلی RIP عملاً نادیده گرفته شود.

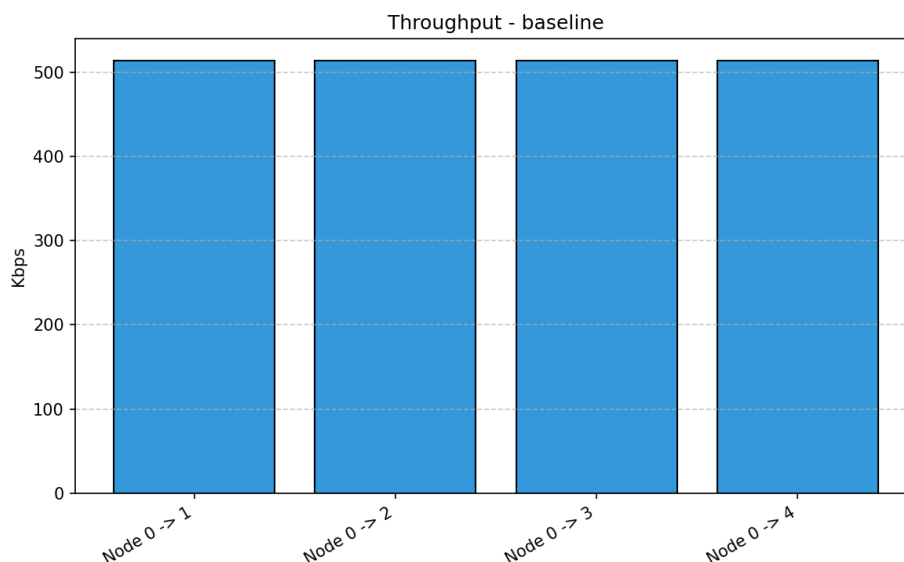
سه معیار اصلی به این صورت محاسبه می‌شوند: PDR (Packet Delivery Ratio) به صورت نسبت بسته‌های دریافت شده به بسته‌های ارسال شده ضرب در صد؛ throughput به صورت نسبت تعداد بیت‌های دریافت شده به مدت زمان مؤثر انتقال (تفاضل زمان آخرین دریافت و اولین ارسال، که معیاری دقیق‌تر از تقسیم بر کل مدت شبیه‌سازی است، چون تنها بازه‌ی واقعاً فعال انتقال را در نظر می‌گیرد)؛ و تأخیر متوسط end-to-end به صورت میانگین مجموع تأخیر تمام بسته‌های دریافتی. تمام این فرمول‌ها شامل بررسی مخرج صفر (division zero by guard) هستند تا در صورتی که یک جریان هیچ بسته‌ای دریافت نکند (مانند گره‌های دور دست در سناریوی scalability)، برنامه دچار خطای محاسباتی (مثل تقسیم بر صفر که می‌تواند مقدار NaN تولید کند) نشود. در نهایت نیز فضای اختصاص داده شده به شبیه‌ساز با دستور Destroy پاکسازی می‌شود تا از نشت حافظه در اجراهای متوالی جلوگیری شود.

۴ شبیه‌سازی، نتایج و ارزیابی عملکرد

برای استخراج گراف‌های این فصل، خروجی `results-<scenario>.xml` هر یک از سه اجرا پردازش شده و نمودارهای میله‌ای throughput، تأخیر، و PDR برای هر جریان رسم شده‌اند. در تمامی سناریوها، گره‌ها در یک ساختار خطی (Linear) با فاصله ۸۰ متر از یکدیگر قرار گرفته‌اند تا ارسال چندگامی (Multi-hop) و عملکرد پروتکل مسیریابی به درستی ارزیابی شود؛ این انتخاب توپولوژیک عمدی، حالت بدترین سناریوی ممکن از نظر قطر شبکه را شبیه‌سازی می‌کند، چون در یک ساختار خطی، فاصله‌ی گام‌محور بین دورترین گره‌ها برابر $n - 1$ است که بزرگ‌ترین مقدار ممکن برای n گره‌ی معین محسوب می‌شود.

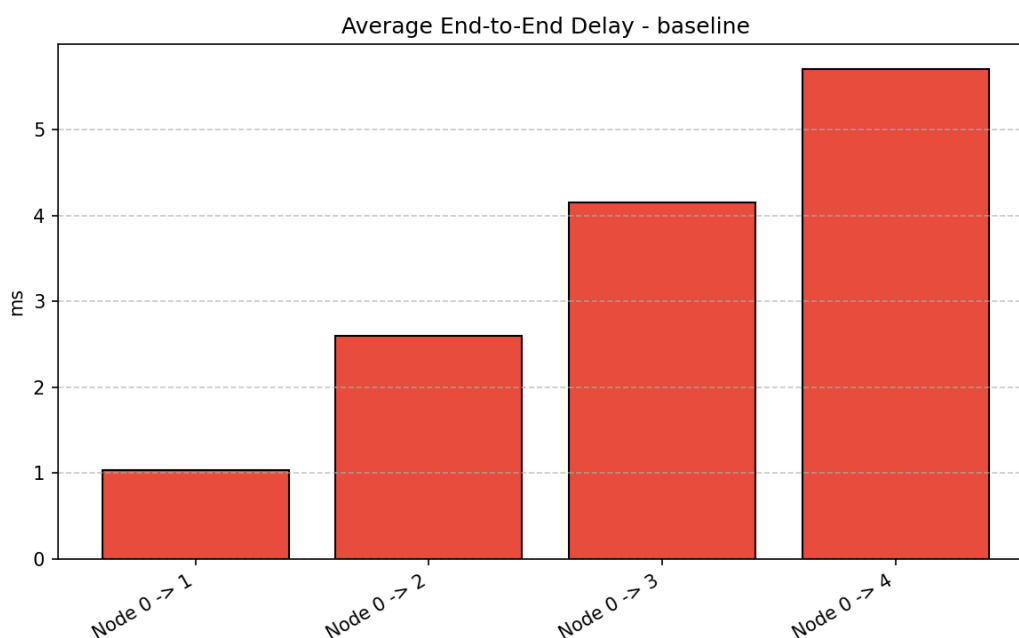
۱.۴ سناریوی پایه: (Baseline) پنج گره، نرخ ترافیک متوسط

در این سناریو، شبکه شامل ۵ گره است و گره مبدأ (گره ۰) ترافیکی با نرخ 500kbps به سمت ۴ گره دیگر ارسال می‌کند.



شکل ۱: نمودار توزیع throughput در سناریوی baseline

همان‌طور که در شکل ۱ مشخص است، throughput تمامی چهار جریان به مقدار نامی 500 کیلو بیت بر ثانیه نزدیک است و به حدود 510 کیلو بیت بر ثانیه (با احتساب سربار هدرهای IP/UDP و لایه‌ی MAC) رسیده است. از آنجا که نرخ تزریق ترافیک در حد تحمل شبکه است، CSMA/CA به خوبی توانسته دسترسی به رسانه را بدون رقابت مخرب مدیریت کند و تقریباً تمام پهنای باند درخواستی به مقصد رسیده است.

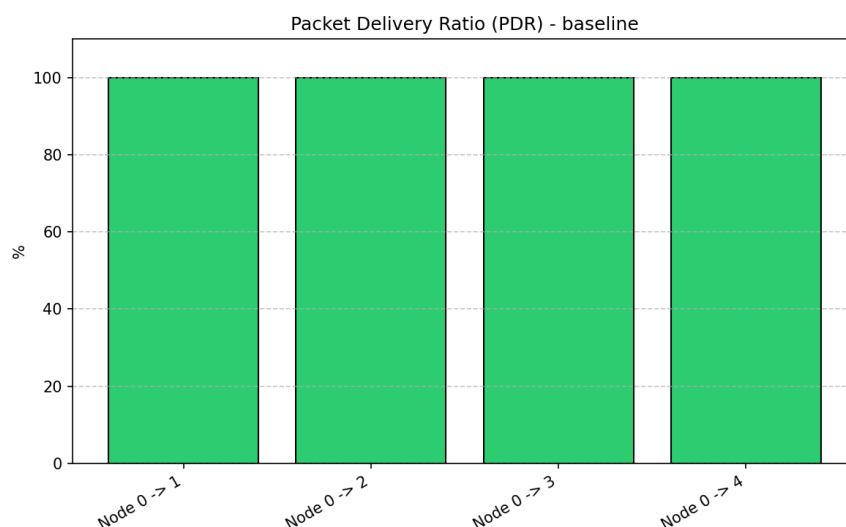


شکل ۲: نمودار میانگین تأخیر end-to-end به شکل پله‌ای در سناریوی baseline

مهم‌ترین نکته در شکل ۲، رفتار کاملاً پله‌ای (staircase) تأخیر است. از آنجا که توپولوژی خطی است، گره ۱ در فاصله یک گام (Hop) و گره ۴ در فاصله چهار گام قرار دارد. هر گام اضافه نیازمند دریافت بسته در لایه‌ی فیزیکی، پردازش جدول مسیریابی RIP برای یافتن next-hop، صف‌بندی مجدد در گره‌ی واسطه، و رقابت مجدد در لایه‌ی MAC برای دسترسی به کانال جهت ارسال



به گام بعدی است. به همین دلیل، تأخیر از حدود ۱ میلی ثانیه برای گره ۱ (تک‌گامی)، به صورت تقریباً خطی تا حدود ۵.۵ میلی ثانیه برای گره ۴ (چهارگامی) انباشته شده است؛ این الگو رابطه‌ی تقریباً خطی بین تأخیر و تعداد گام‌ها را به خوبی تأیید می‌کند و نشان می‌دهد که در شرایط بدون اشباع، هر گام افزوده‌ای سهم تقریباً ثابتی (حدود ۱ تا ۵.۱ میلی ثانیه) به تأخیر کل اضافه می‌کند.

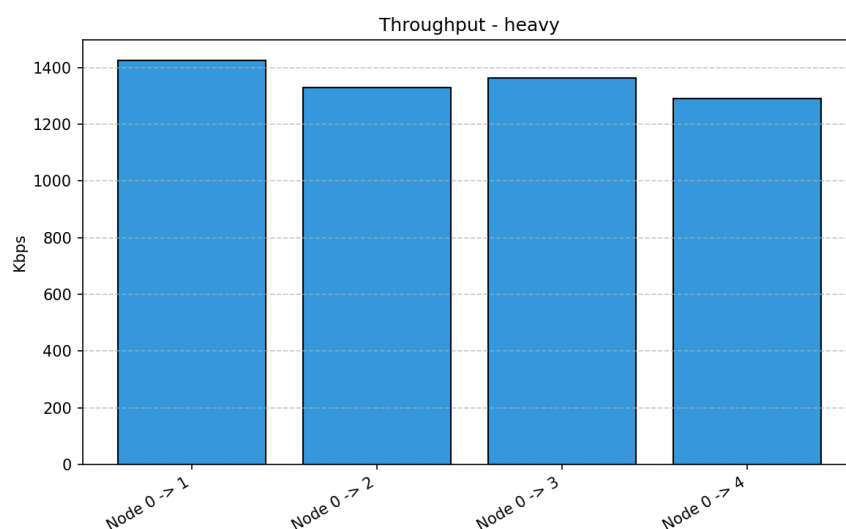


شکل ۳: نمودار Ratio Delivery Packet در سناریوی baseline

در این شرایط ایده‌آل و غیراشباع، نرخ تحویل بسته (PDR) برای تمامی گره‌ها دقیقاً ۱۰۰ درصد است (شکل ۳) و مسیریابی RIP و انتقال MAC بدون از دست رفتن بسته‌ها انجام شده است؛ این نتیجه نشان می‌دهد که در بار سبک، نه ازدحام MAC و نه محدودیت همگرایی، RIP هیچ‌کدام مانعی برای تحویل کامل ترافیک ایجاد نمی‌کنند.

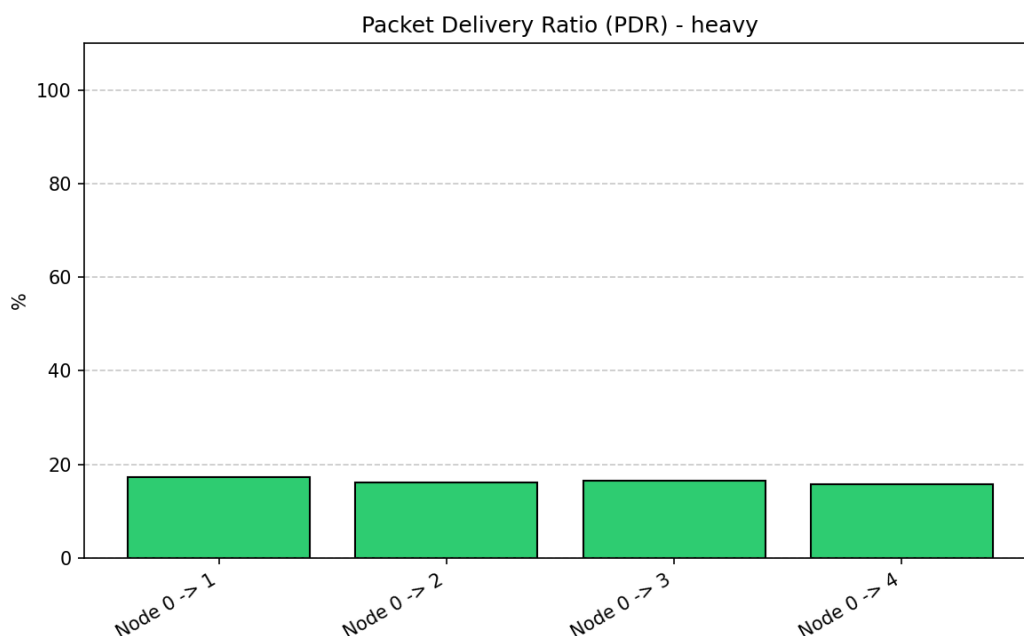
۲.۴ سناریوی ترافیک سنگین (Heavy Traffic)

در این سناریو نرخ تزریق ترافیک به شدت افزایش یافته تا شبکه به حالت اشباع (Saturation) برسد.



شکل ۴: نمودار throughput در سناریوی ترافیک سنگین

در شکل ۴ مشاهده می‌شود که throughput هر جریان روی مقداری بین 1300 تا 1400 کیلو بیت بر ثانیه متوقف شده است، به‌رغم این که نرخ تزریق درخواستی به‌مراتب بالاتر از این مقدار است. این سقف، نمایانگر ظرفیت مؤثر (Effective Capacity) کانال $802.11b$ در این توپولوژی چندگامی است؛ این مقدار به‌طور قابل‌توجهی پایین‌تر از نرخ نظری فیزیکی 11 مگابیت بر ثانیه است، چون باید بین چهار جریان هم‌زمان و چندین گام میانی تقسیم شود و سربار ناشی از $DIFS$ ، $backoff$ و ACK نیز از ظرفیت مفید کانال می‌کاهد. این پدیده نمونه‌ی کلاسیکی از پدیده‌ی شناخته‌شده‌ی «کاهش ظرفیت چندگامی» (multi-hop capacity degradation) در شبکه‌های بی‌سیم ad-hoc است، که طبق آن ظرفیت end-to-end یک مسیر h گامی می‌تواند به نسبت تقریبی $1/h$ نسبت به ظرفیت یک گامی کاهش یابد، چون هر گام میانی هم باید بسته را دریافت و هم دوباره ارسال کند و این دو عمل بر سر همان کانال رادیویی مشترک رقابت می‌کنند.

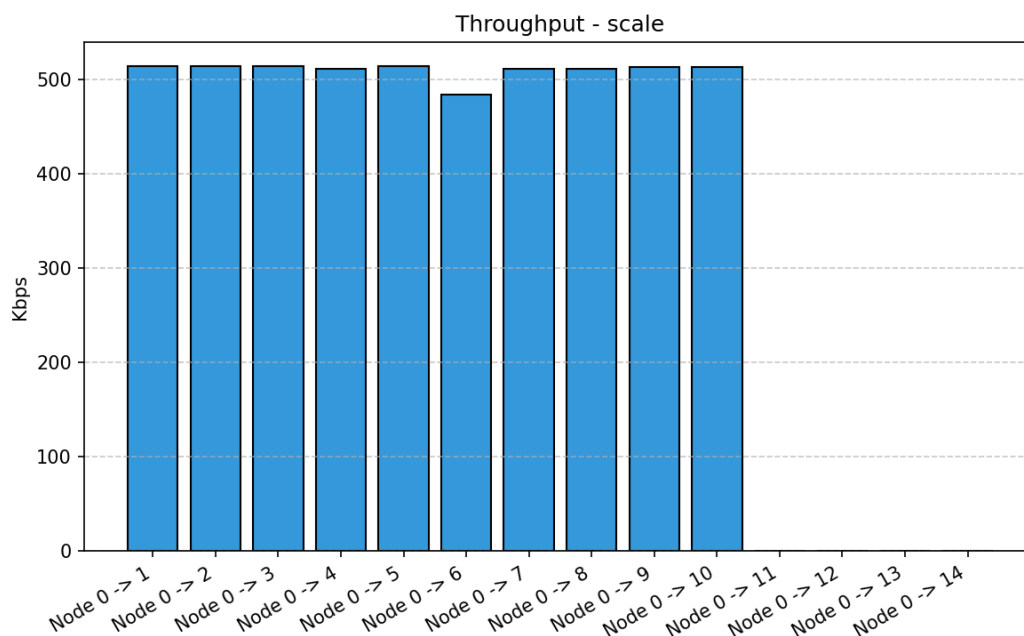


شکل ۵: سقوط شدید Ratio Delivery Packet در سناریوی ترافیک سنگین

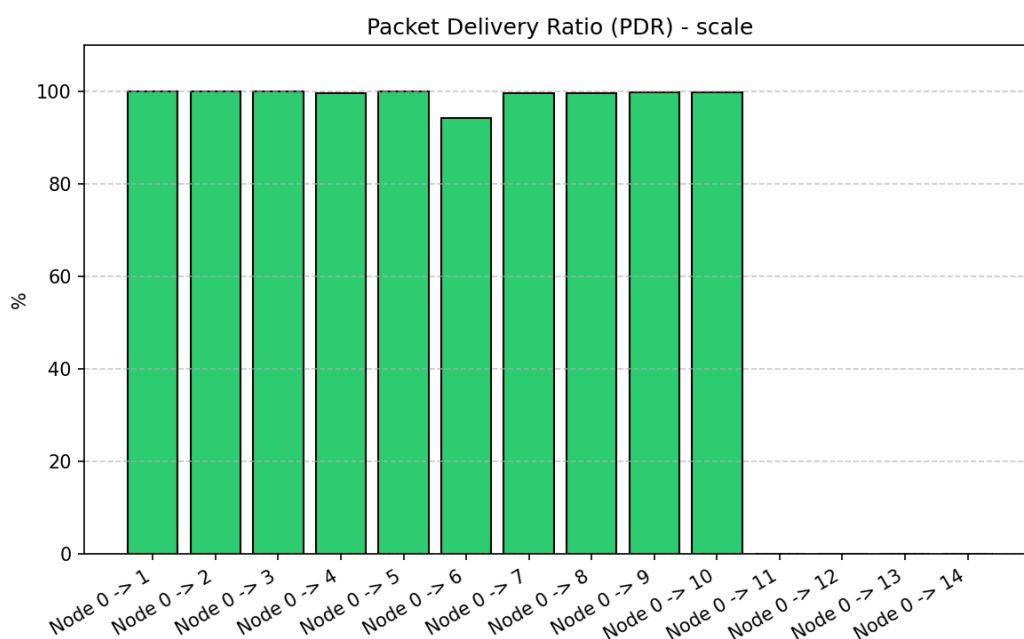
بارزترین خروجی این سناریو، سقوط شدید PDR به زیر 20% درصد برای تمامی جریان‌هاست (شکل ۵). در توپولوژی خطی، ترافیک بالا باعث ایجاد تداخل شدید درون‌جریانی (Intra-flow Interference) و بروز پدیده گره پنهان (Hidden Node Problem) می‌شود. مکانیزم عقب‌نشینی (Backoff) در CSMA/CA توانایی مدیریت این حجم از برخوردها را ندارد و بخش عمده‌ای از بسته‌ها به دلیل سرریز شدن صف‌های ارسال (تنظیم پیش‌فرض اندازه‌ی صف در $3NS$ محدود است) یا پایان یافتن محدودیت تلاش مجدد MAC (که پس از تعداد مشخصی retransmission ناموفق، بسته را به‌طور کامل dropped می‌کند)، از بین می‌روند. این افت شدید PDR در کنار محدود ماندن throughput روی یک سقف ثابت، به‌خوبی نشان می‌دهد که اشباع کانال نه‌فقط باعث کاهش بازده، بلکه باعث اتلاف واقعی منابع شبکه (تلاش‌های ارسال ناموفق که خود نیز کانال را اشغال می‌کنند) نیز می‌شود.

۳.۴ سناریوی مقیاس‌پذیری: (Scalability) افزایش تعداد گره‌ها

در این سناریو شبکه به 15 گره در یک ساختار خطی گسترش یافته است تا محدودیت‌های پروتکل‌ها محک زده شود. نرخ ترافیک همان 500kbps است، یعنی بار سبک و قابل‌مدیریت برای CSMA/CA حفظ شده تا بتوان اثر افزایش قطر شبکه را به‌طور مجزا (بدون آلودگی با اثر اشباع کانال) بررسی کرد.



شکل ۶: صفر شدن Throughput گره‌های انتهایی در سناریوی مقیاس‌پذیری



شکل ۷: تغییرات PDR و قطع ارتباط گره‌های دورتر در سناریوی مقیاس‌پذیری

برخلاف تئوری افت یکنواخت پهنای باند، نمودارهای ۶ و ۷ یک نقطه ضعف بنیادین در پروتکل RIP را آشکار می‌کنند، نه یک افت تدریجی ناشی از تداخل. MAC گره‌های ۱ تا ۱۰ توانسته‌اند ترافیک خود را با PDR نزدیک به ۱۰۰٪ دریافت کنند (با یک افت جزئی در گره ۶ که به عنوان تنگنا یا Bottleneck میانی زنجیره عمل کرده است، چون این گره باید هم‌زمان نقش بازپخش (relay) برای ترافیک گره‌های دورتر را ایفا کند و هم ترافیک خودش را دریافت کند). اما PDR و Throughput برای گره‌های ۱۱ تا ۱۴ کاملاً صفر است؛ این یک افت ناگهانی (cliff-edge) است، نه یک کاهش تدریجی.



دلیل این امر، زمان طولانی همگرایی (Convergence Time) در پروتکل RIP است. انتشار دوره‌ای جداول مسیریابی در یک زنجیره‌ی طولانی به زمان بسیار زیادی نیاز دارد، چون اطلاع‌رسانی یک مسیر جدید باید گام به گام تا انتهای زنجیره منتشر شود و در هر گام، علاوه بر تأخیر انتشار، باید منتظر دوره‌ی بعدی broadcast دوره‌ای RIP نیز ماند. در نتیجه، پیش از پایان یافتن فاصله‌ی زمانی اختصاص یافته برای همگرایی، RIP فرصت نکرده مسیر معتبری برای گره‌های انتهایی (۱۱ تا ۱۴) پیدا کند. در نتیجه، بسته‌های مربوط به این گره‌ها به دلیل نداشتن مسیر (No Route to Host) همان در گره مبدأ (یا اولین گره‌ی میانی فاقد مسیر) دراپ شده‌اند، بدون این‌که حتی فرصتی برای ورود به رقابت MAC داشته باشند. این نتیجه به‌روشنی نشان می‌دهد که در این سناریوی خاص، گلوگاه عملکرد، لایه‌ی شبکه (RIP) است، نه لایه‌ی MAC.

۴.۴ مقایسه‌ی تطبیقی سه سناریو

جدول ۲: جمع‌بندی مقایسه‌ای کیفی سه سناریوی شبیه‌سازی شده

Scalability	Traffic Heavy	Baseline	معیار
۱۵ 500kbps	۵ اشباع کننده	۵ 500kbps	تعداد گره‌ها
غیراشباع برای نودهای ابتدایی	اشباع شده	غیراشباع	نرخ هر جریان
صفر برای نودهای انتهایی	محدود به ظرفیت عملی	تحویل کامل	وضعیت کانال
پله‌ای افزایشی	بسیار بالا	پله‌ای (وابسته به گام)	throughput
۱۰۰٪ برای همسایگان، ۰٪ برای دور دست	افت شدید (زیر ۲۰٪)	۱۰۰٪	تأخیر متوسط
کندی همگرایی RIP	Node Hidden و تداخل MAC	-	PDR
			چالش اصلی

این جدول به خوبی نشان می‌دهد که علت اصلی افت عملکرد در هر سناریو کاملاً متفاوت است: در traffic، heavy، گلوگاه در لایه‌ی MAC و ظرفیت فیزیکی کانال است؛ در حالی که در scalability، گلوگاه در لایه‌ی شبکه و سرعت محدود پروتکل مسیریابی distance-vector است. این تفکیک علت‌ها، اهمیت طراحی آزمایش با کنترل دقیق متغیرها (تغییر یک پارامتر در هر سناریو، با ثابت نگه داشتن بقیه) را نشان می‌دهد.

۵ پاسخ به سوالات پروژه

۱.۵ چگونگی مدیریت ارسال‌های هم‌زمان توسط CSMA/CA و دلیل برتری آن نسبت به CS-MA/CD در محیط بی‌سیم

CSMA/CA با استفاده از مکانیزم DCF ارسال هم‌زمان را با ترکیبی از حس کردن کانال پیش از ارسال (Carrier Sense)، فاصله‌ی زمانی اجباری DIFS و backoff تصادفی از یک Window Contention مدیریت می‌کند؛ هر گره پیش از ارسال باید کانال را برای مدت DIFS آزاد ببیند و سپس یک تایمر backoff تصادفی را شمارش معکوس کند که در صورت مشغول شدن کانال در میانه‌ی راه، متوقف (freeze) و پس از آزاد شدن مجدد کانال ادامه می‌یابد. در نتایج این پروژه، این مکانیزم در سناریوی baseline به خوبی توانست چهار جریان هم‌زمان را بدون افت PDR مدیریت کند (PDR صد درصد)، اما در سناریوی traffic heavy که نرخ ترافیک از ظرفیت مؤثر کانال فراتر رفت، backoff نمایی توانست throughput را روی یک سقف نسبتاً پایدار نگه دارد، هرچند با هزینه‌ی افت شدید PDR و افزایش چشمگیر تأخیر.

دلیل برتری CSMA/CA نسبت به CSMA/CD در محیط بی‌سیم، به دو محدودیت فیزیکی بنیادین بازمی‌گردد. نخست، رادیوهای معمولی Wi-Fi به صورت half-duplex عمل می‌کنند و توان سیگنال ارسالی یک گره به مراتب از توان سیگنال دریافتی است؛ بنابراین گیرنده‌ی همان گره عملاً قادر به تشخیص یک تداخل ضعیف از سوی گره‌ی دیگر در حین ارسال خودش نیست (سیگنال

خودش signal دریافتی‌های دیگر را می‌پوشاند). دوم، مسئله‌ی terminal hidden باعث می‌شود دو گره که هر دو در محدوده‌ی یک گیرنده‌ی مشترک هستند، اما خودشان از محدوده‌ی رادیویی یکدیگر خارج باشند، اصلاً از ارسال هم‌زمان یکدیگر مطلع نشوند. این دو محدودیت در شبکه‌های سیمی (که در آن‌ها CSMA/CD کاربرد دارد) به دلیل ساختار اشتراکی و متقارن کابل وجود ندارند. به همین دلیل، 802.11 به جای رویکرد «تشخیص پس از وقوع» (Collision Detection)، رویکرد «پیشگیری پیش از وقوع» (Collision Avoidance) را برگزیده و از مکانیزم تأییدیه‌ی صریح (ACK) برای جبران ناتوانی در تشخیص مستقیم collision استفاده می‌کند.

۲.۵ مسیرهای کشف‌شده توسط RIP و سرعت برقراری آن‌ها

با توجه به توپولوژی خطی به کاررفته در این پروژه، RIP مسیرهایی متناظر با زنجیره‌ی گام‌به‌گام بین گره‌ها کشف کرده است: برای رسیدن از گره o به گره k ، مسیر بهینه (و در این توپولوژی، تنها مسیر ممکن) از طریق گره‌های میانی $1, 2, \dots, k-1$ عبور می‌کند، چون این توپولوژی برخلاف یک grid کامل، فاقد مسیرهای جایگزین (alternate paths) است. الگوریتم Bellman-Ford توزیع‌شده‌ی RIP این مسیرها را با تبادل دوره‌ای بردارهای فاصله بین همسایگان مستقیم کشف می‌کند: هر گره فاصله‌ی خود تا هر مقصد را به همسایگان‌اش اعلام می‌کند و همسایه این فاصله را به علاوه‌ی یک گام در نظر می‌گیرد.

سرعت برقراری این مسیرها مستقیماً به قطر شبکه وابسته است. در سناریوی baseline با ۵ گره (قطر ۴ گام)، فاصله‌ی ۳۴ ثانیه‌ای پیش از شروع ترافیک داده برای همگرایی کامل کافی بوده و نتیجه‌ی آن PDR صد درصد برای تمام جریان‌ها بوده است؛ این نشان می‌دهد که RIP در این مقیاس کوچک، طی چند دور تبادل پیام (به مراتب کمتر از زمان اختصاص‌یافته) به همگرایی کامل رسیده است. اما در سناریوی scalability با ۱۵ گره (قطر ۱۴ گام)، همان فاصله‌ی زمانی برای همگرایی کافی نبوده است: نتایج نشان می‌دهند که مسیرهای گره‌های ۱ تا ۱۰ به درستی برقرار شده‌اند (با PDR نزدیک به ۱۰۰٪)، اما مسیرهای گره‌های ۱۱ تا ۱۴ هرگز در جداول مسیریابی برقرار نشدند (PDR صفر) و بسته‌های مربوط به آن‌ها به دلیل «عدم وجود مسیر» در همان گره‌ی مبدأ یا نزدیک به آن دراپ شدند. این رفتار به وضوح نشان‌دهنده‌ی رابطه‌ی غیرخطی بین قطر شبکه و زمان همگرایی RIP است که در شبکه‌های بزرگ‌تر می‌تواند به یک محدودیت اساسی تبدیل شود.

۳.۵ تفاوت عملکرد بین چهار جریان ترافیکی

در سناریوی baseline، چهار جریان ترافیکی (به سمت گره‌های ۱ تا ۴) تفاوت معناداری در throughput و PDR نشان نمی‌دهند؛ همگی به throughput نزدیک به نرخ نامی ۵۰۰ کیلو بیت بر ثانیه و PDR صد درصد می‌رسند، چون بار کلی کانال هنوز در محدوده‌ی غیراشباع قرار دارد. اما تفاوت اصلی و سیستماتیک بین این چهار جریان، در معیار تأخیر end-to-end ظاهر می‌شود: تأخیر با تعداد گام‌های لازم برای رسیدن به هر مقصد، به صورت تقریباً خطی افزایش می‌یابد. جریان به سمت گره ۱ (تک‌گامی) کمترین تأخیر (حدود ۱ میلی‌ثانیه) و جریان به سمت گره ۴ (چهارگامی) بیشترین تأخیر (حدود ۵.۵ میلی‌ثانیه) را دارد، چون هر گام میانی، تأخیر صف‌بندی و رقابت MAC مستقل خود را به تأخیر تجمعی اضافه می‌کند. در سناریوی heavy traffic، این تفاوت به مراتب تشدید می‌شود، چون با اشباع کانال، تأخیر صف‌بندی در هر گام به طور نمایی (نه خطی) افزایش می‌یابد و جریان‌های با گام بیشتر، احتمال بیشتری برای drop شدن بسته‌هایشان در گره‌های میانی دارند، نه فقط در گام نهایی.

۴.۵ رفتار شبکه با تعداد گره‌ی بیشتر یا ترافیک سنگین‌تر

با افزایش حجم ترافیک (سناریوی heavy traffic)، شبکه به نقطه‌ی اشباع کانال می‌رسد: throughput تجمعی روی یک سقف ثابت (ظرفیت مؤثر کانال 802.11b در این توپولوژی چندگامی، که به مراتب کمتر از نرخ نظری ۱۱ مگابیت بر ثانیه است) متوقف می‌شود، در حالی که PDR به شدت سقوط می‌کند (به زیر ۲۰٪)، چون مکانیزم backoff نمایی دیگر قادر به مدیریت حجم بالای رقابت نیست و درصد بالایی از بسته‌ها پس از اتمام تلاش‌های مجدد مجاز، به طور کامل از بین می‌روند. این رفتار نشان‌دهنده‌ی فروپاشی کارایی (performance collapse) معمول در شبکه‌های CSMA تحت بار بیش از ظرفیت است. با افزایش تعداد گره‌ها در یک نرخ ترافیک ثابت و سبک (سناریوی scalability)، اما رفتار شبکه به طور کیفی متفاوت است: گلوگاه دیگر MAC نیست، بلکه سرعت همگرایی RIP است. گره‌های نزدیک به مبدأ، عملکرد کامل (PDR ۱۰۰٪) را حفظ می‌کنند، اما

از یک نقطه‌ی مشخص به بعد (در این پروژه، حدود گام دهم تا یازدهم)، PDR به‌طور ناگهانی به صفر سقوط می‌کند، نه به‌صورت تدریجی. این یک افت آستانه‌ای (threshold effect) است، نه یک کاهش پیوسته، و علت آن کاملاً متفاوت از علت افت در سناریوی traffic heavy است.

۵.۵ محدودیت‌های احتمالی RIP و CSMA/CA در مقیاس بزرگ‌تر

محدودیت‌های RIP در مقیاس بزرگ شامل موارد زیر است: نخست، سرعت همگرایی کند که با قطر شبکه به‌صورت خطی (و در عمل، به‌دلیل وابستگی به دوره‌ی broadcast ثابت، حتی کندتر) افزایش می‌یابد؛ نتایج این پروژه دقیقاً همین محدودیت را برای شبکه‌ای با قطر ۱۴ گام نشان داد. دوم، محدودیت hop-count برابر ۱۶ به‌عنوان «بی‌نهایت»، که سقفی برای حداکثر قطر شبکه‌ی قابل پشتیبانی توسط RIP تحمیل می‌کند؛ شبکه‌هایی با قطر بیشتر از ۱۵ گام اساساً قابل مسیریابی صحیح با RIP نیستند. سوم، مشکل count-to-infinity در صورت قطع لینک، که می‌تواند زمان همگرایی مجدد پس از یک تغییر توپولوژی را به‌طور چشمگیری افزایش دهد. این محدودیت‌ها به‌طور کلی RIP را برای شبکه‌های ad-hoc بزرگ یا با تغییرات توپولوژیکی مکرر، نامناسب می‌سازند و استفاده از پروتکل‌های link-state (مانند OSPF) یا پروتکل‌های مخصوص شبکه‌های ad-hoc (مانند AODV یا OLSR، که برای مقابله با تحرک و تغییرات سریع توپولوژی طراحی شده‌اند) را در مقیاس بزرگ‌تر توجیه می‌کند.

محدودیت‌های CSMA/CA در مقیاس بزرگ‌تر عمدتاً حول مسئله‌ی ظرفیت ثابت کانال مشترک می‌چرخد: با افزایش تعداد گره‌هایی که هم‌زمان در محدوده‌ی رادیویی یکدیگر قرار دارند یا با افزایش تعداد جریان‌های هم‌زمان از یک گره (مانند طراحی این پروژه که گره صفر باید با تمام گره‌های دیگر هم‌زمان ارتباط برقرار کند)، رقابت بر سر کانال به‌صورت نمایی افزایش می‌یابد و throughput عادلانه (fair throughput) به ازای هر جریان کاهش می‌یابد؛ این پدیده با نتایج traffic heavy در این پروژه به‌خوبی نشان داده شد. علاوه بر این، در توپولوژی‌های چندگامی بزرگ‌تر، مسئله‌ی terminal hidden با افزایش تعداد گره‌ها تشدید می‌شود، چون احتمال وجود جفت‌گره‌هایی که هم‌زمان در محدوده‌ی یک گیرنده‌ی مشترک باشند اما خارج از محدوده‌ی شنیداری یکدیگر باشند، با اندازه‌ی شبکه افزایش می‌یابد. در نهایت، هر گام میانی در یک مسیر طولانی، علاوه بر تحمیل تأخیر تجمعی (که در نتایج baseline این پروژه به‌وضوح دیده شد)، عملاً ظرفیت کانال را نیز با خود ارسال (relay) بیشتر مصرف می‌کند که به کاهش ظرفیت end-to-end با نسبت تقریبی $1/h$ منجر می‌شود، که هر دو محدودیت RIP و CSMA/CA با افزایش مقیاس شبکه، به‌طور هم‌زمان و تشدیدکننده‌ی یکدیگر عمل می‌کنند.